

第 6 章 PCIe 总线的事务层

事务层是 PCIe 总线层次结构的最高层，该层次将接收 PCIe 设备核心层的数据请求，并将其转换为 PCIe 总线事务，PCIe 总线使用的这些总线事务在 TLP 头中定义。PCIe 总线继承了 PCI/PCI-X 总线的大多数总线事务，如存储器读写、I/O 读写、配置读写总线事务，并增加了 Message 总线事务和原子操作等总线事务。

本节重点介绍与数据传送密切相关的总线事务，如存储器、I/O、配置读写总线事务。在 PCIe 总线中，Non-Posted 总线事务分两部分进行，首先是发送端向接收端提交总线读写请求，之后接收端再向发送端发送完成（Completion）报文。PCIe 总线使用 Split 传送方式处理所有 Non-Posted 总线事务，存储器读、I/O 读写和配置读写这些 Non-Posted 总线事务都使用 Split 传送方式。PCIe 的事务层还支持流量控制和虚通路管理等一系列特性，而 PCI 总线并不支持这些新的特性。

在 PCIe 总线中，不同的总线事务采用的路由方式不相同。PCIe 总线继承了 PCI 总线的地址路由和 ID 路由方式，并添加了“隐式路由”方式。

PCIe 总线使用的数据报文首先在事务层中形成，这个数据报文也称为事务层数据报文，即 TLP。TLP 在经过数据链路层时被加上 Sequence Number 前缀和 CRC 后缀，然后发向物理层。

数据链路层还可以产生 DLLP（Data Link Layer Packet）。DLLP 和 TLP 没有直接关系。DLLP 是产生于数据链路层，终止于数据链路层，并不会传递到事务层。DLLP 不是 TLP 加上前缀和后缀形成的。数据链路层的报文 DLLP 通过物理层时，需要经过 8/10b 编码，然后再进行发送。而数据的接收过程是发送过程的逆过程。

6.1 TLP 的格式

当处理器或者其他 PCIe 设备访问 PCIe 设备时，所传送的数据报文首先通过事务层被封装为一个或者多个 TLP，之后才能通过 PCIe 总线的各个层次发送出去。TLP 的基本格式如图 6-1 所示。



图 6-1 TLP 的基本格式

一个完整的 TLP 由 1 个或者多个 TLP Prefix、TLP Head、Data Payload（数据有效负载）和 TLP Digest 组成。TLP 头是 TLP 最重要的标志，不同的 TLP 其头的定义并不相同。TLP 头包含了当前 TLP 的总线事务类型、路由信息等一系列信息。在一个 TLP 中，Data Payload 的长度可变，最小为 0，最大为 1024DW。

TLP Digest 是一个可选项，一个 TLP 是否需要 TLP Digest 由 TLP 头决定。Data Payload 也是一个可选项，有些 TLP 并不需要 Data Payload，如存储器读请求、配置和 I/O 写完成 TLP 并不需要 Data Payload。

TLP Prefix 由 PCIe V2.1 总线规范引入，分为 Local TLP Prefix 和 EP-EP TLP Prefix 两类。其中 Local TLP Prefix 的主要作用是在 PCIe 链路的两端传递消息，而 EP-EP TLP Prefix 的主要作用是在发送设备和接收设备之间传递消息。设置 TLP Prefix 的主要目的是为了扩展 TLP 头，并以此支持 PCIe V2.1 规范的一些新的功能。

TLP 头由 3 个或者 4 个双字（DW）组成。其中第一个双字中保存通用 TLP 头，其他字段与通用 TLP 头的 Type 字段相关。一个通用 TLP 头由 Fmt、Type、TC、Length 等字段组成，如图 6-2 所示。



图 6-2 通用 TLP 头格式

如果存储器读写 TLP 支持 64 位地址模式时，TLP 头的长度为 4DW，否则为 3DW。而完成报文的 TLP 头不含有地址信息，使用的 TLP 头长度为 3DW。其中 Byte 4~Byte 15 的格式与 TLP 相关，下文将结合具体的 TLP 介绍这些字段。

6.1.1 通用 TLP 头的 Fmt 字段和 Type 字段

Fmt 和 Type 字段确认当前 TLP 使用的总线事务，TLP 头的大小是由 3 个双字还是 4 个双字组成，当前 TLP 是否包含有效负载。其具体含义如表 6-1 所示。

表 6-1 Fmt[2:0] 字段

Fmt[2:0]	TLP 的格式
0b000	TLP 大小为 3 个双字，不带数据
0b001	TLP 大小为 4 个双字，不带数据
0b010	TLP 大小为 3 个双字，带数据
0b011	TLP 大小为 4 个双字，带数据
0b100	TLP Prefix
其他	PCIe 总线保留

其中所有读请求 TLP 都不带数据，而写请求 TLP 带数据，而其他 TLP 可能带数据也可能不带数据，如完成报文可能含有数据，也可能仅含有完成标志而并不携带数据。在 TLP 的 Type 字段中存放 TLP 的类型，即 PCIe 总线支持的总线事务。该字段共由 5 位组成，其含

义如表 6-2 所示。

表 6-2 Type[4:0] 字段

TLP 类型	Fmt[2:0]	Type[4:0]	描 述
MRd	0b000 0b001	0b0 0000	存储器读请求；TLP 头大小为 3 个或者 4 个双字，不带数据
MRdLk	0b000 0b001	0b0 0001	带锁的存储器读请求；TLP 头大小为 3 个或者 4 个双字，不带数据
MWr	0b010 0b011	0b0 0000	存储器写请求；TLP 头大小为 3 个或者 4 个双字，带数据
IOrd	0b000	0b0 0010	IO 读请求；TLP 头大小为 3 个双字，不带数据
IOWr	0b010	0b0 0010	IO 写请求；TLP 头大小为 3 个双字，带数据
CfgRd0	0b000	0b0 0100	配置 0 读请求；TLP 头大小为 3 个双字，不带数据
CfgWr0	0b010	0b0 0100	配置 0 写请求；TLP 头大小为 3 个双字，带数据
CfgRd1	0b000	0b0 0101	配置 1 读请求；不带数据
CfgWr1	0b010	0b0 0101	配置 1 写请求；带数据
TCfgRd	0b010	0b1 1011	本书对这两种总线事务不做介绍
TCfgWr	0b001	0b1 1011	
Msg	0b001	0b1 0r ₂ r ₁ r ₀	消息请求；TLP 头大小为 4 个双字，不带数据。“rrr” 字段是消息请求报文的 Route 字段，下文将详细介绍该字段
MsgD	0b011	0b1 0r ₂ r ₁ r ₀	消息请求；TLP 头大小为 4 个双字，带数据
Cpl	0b000	0b0 1010	完成报文；TLP 头大小为 3 个双字，不带数据。包括存储器、配置和 I/O 写完成
CplD	0b001	0b0 1010	带数据的完成报文，TLP 头大小为 3 个双字，包括存储器读、I/O 读、配置读和原子操作读完成
CplLk	0b000	0b0 1011	锁定的完成报文，TLP 头大小为 3 个双字，不带数据
CplDLk	0b010	0b0 1011	带数据的锁定完成报文，TLP 头大小为 3 个双字，带数据
FetchAdd	0b010 0b011	0b0 1100	Fetch and Add 原子操作
Swap	0b010 0b011	0b0 1101	Swap 原子操作
CAS	0b010 0b011	0b0 1110	CAS 原子操作
LPrfx	0b100	0b0 L ₃ L ₂ L ₁ L ₀	Local TLP Prefix
EPrfx	0b100	0b1 E ₃ E ₂ E ₁ E ₀	End-End TLP Prefix

存储器读和写请求，IO 读和写请求，及配置读和写请求的 type 字段相同，如存储器读和写请求的 Type 字段都为 0b0 0000。此时 PCIe 总线规范使用 Fmt 字段区分读写请求，当 Fmt 字段是“带数据”的报文，一定是“写报文”；当 Fmt 字段是“不带数据”的报文，一定是“读报文”。

PCIe 总线的数据报文传送方式与 PCI 总线数据传送有类似之处。其中存储器写 TLP 使用 Posted 方式进行传送，而其他总线事务使用 Non-Posted 方式。

PCIe 总线规定所有 Non-Posted 存储器请求使用 Split 总线方式进行数据传递。当 PCIe 设备进行存储器读、I/O 读写或者配置读写请求时，首先向目标设备发送数据读写请求 TLP，当目标设备收到这些读写请求 TLP 后，将数据和完成信息通过完成报文（Cpl 或者 CplD）发送给源设备。

其中存储器读、I/O 读和配置读需要使用 CplD 报文，因为目标设备需要将数据传递给源设备；而 I/O 写和配置写需要使用 Cpl 报文，因为目标设备不需要将任何数据传递给源设备，但是需要通知源设备，写操作已经完成，数据已经成功地传递给目标设备。

在 PCIe 总线中，进行存储器或者 I/O 写操作时，数据与数据包头一起传递；而进行存储器或者 I/O 读操作时，源设备首先向目标设备发送读请求 TLP，而目标设备在准备好数据后，向源设备发出完成报文。

PCIe 总线规范还定义了 MRdLk 报文，该报文的主要作用是与 PCI 总线的锁操作相兼容，但是 PCIe 总线规范并不建议用户使用这种功能，因为使用这种功能将极大影响 PCIe 总线的数据传送效率。

与 PCI 总线不同，PCIe 总线规范定义了 Msg 报文，即消息报文，分别为 Msg 和 MsgD。这两种报文的区别在于一个报文可以传递数据，一个不能传递数据。

PCIe V2.1 总线规范还补充了一些总线事务，如 FetchAdd、Swap、CAS、LPrfx 和 EPrfx。其中 LPrfx 和 EPrfx 总线事务分别与 Local TLP Prefix 和 EP-EP TLP Prefix 对应。在 PCIe 总线规范 V2.0 中，TLP 头的大小为 1DW，而使用 LPrfx 和 EPrfx 总线事务可以对 TLP 头进行扩展，本节不对这些 TLP Prefix 做进一步介绍。PCIe 设备可以使用 FetchAdd、Swap 和 CAS 总线事务进行原子操作，第 6.3.5 节将详细介绍该类总线事务。

6.1.2 TC 字段

TC 字段表示当前 TLP 的传送类型，PCIe 总线规定了 8 种传输类型，分别为 TC0~TC7，缺省值为 TC0，该字段与 PCIe 的 QoS 相关。PCIe 设备使用 TC 区分不同类型的数据传递，而多数 EP 中只含有一个 VC，因此这些 EP 在发送 TLP 时，也仅仅使用 TC0，但是有些对实时性要求较高的 EP 中，含有可以设置 TC 字段的寄存器。

在 Intel 的高精度声卡控制器（High Definition Audio Controller）的扩展配置空间中含有一个 TCSEL 寄存器。系统软件可以设置该寄存器，使声卡控制器发出的 TLP 使用合适的 TC。声卡控制器可以使用 TC7 传送一些对实时性要求较强的控制信息，而使用 TC0 传送一般的数据信息。在具体实现中，一个 EP 也可以将控制 TC 字段的寄存器放入到设备的 BAR 空间中，而不必和 Intel 的高精度声卡控制器相同，存放在 PCI 配置空间中。

目前许多处理器系统的 RC 仅支持一个 VC 通路，此时 EP 使用不同的 TC 进行传递数据的意义不大。x86 处理器的 MCH 中一般支持两个 VC 通路，而多数 PowerPC 处理器仅支持一个 VC 通路。PLX 公司的多数 Switch 也仅支持两个 VC 通路。

有些 RC，如 MPC8572 处理器，也能决定其发出 TLP 使用的 TC。在该处理器的 PCIe Outbound 窗口寄存器（PEXOWARn）中，含有一个 TC 字段，通过设置该字段可以确定 RC 发出的 TLP 使用的 TC 字段。不同的 TC 可以使用 PCIe 链路中的不同 VC，而不同的 VC 的仲裁级别并不相同。EP 或者 RC 通过调整其发出 TLP 的 TC 字段，可以调整 TLP 使用的 VC，从而调整 TLP 的优先级。

6.1.3 Attr 字段

Attr 字段由 3 位组成，其中第 2 位表示该 TLP 是否支持 PCIe 总线的 ID-based Ordering；第 1 位表示是否支持 Relaxed Ordering；而第 0 位表示该 TLP 在经过 RC 到达存储器时，是否需要进行 Cache 共享一致性处理。Attr 字段如图 6-3 所示。

一个 TLP 可以同时支持 ID-based Ordering 和 Relaxed Ordering 两种位序。Relaxed Ordering 最早在 PCI-X 总线规范中提出，用来提高 PCI-X 总线的数据传送效率；而 ID-based Ordering 由 PCIe V2.1 总线规范提出。TLP 支持的序如表 6-3 所示。

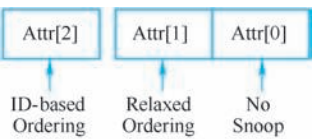


图 6-3 Attr 字段格式

表 6-3 TLP 支持的序

Attr[2]	Attr[1]	类 型
0	0	缺省序，即强序模型
0	1	PCI-X Relaxed Ordering 模型
1	0	ID-based Ordering（IDO）模型
1	1	同时支持 Relaxed Ordering 和 IDO 模型

当使用标准的强序模型时，在数据的整个传送路径中，PCIe 设备在处理相同类型的 TLP 时，如 PCIe 设备发送两个存储器写 TLP 时，后面的存储器写 TLP 必须等待前一个存储器写 TLP 完成后才能被处理，即便当前报文在传送过程中被阻塞，后一个报文也必须等待。

如果使用 Relaxed Ordering 模型，后一个存储器写 TLP 可以穿越前一个存储器写 TLP，提前执行，从而提高了 PCIe 总线的利用率。有时一个 PCIe 设备发出的 TLP，其目的地址并不相同，可能先进入发送队列的 TLP，在某种情况下无法发送，但这并不影响后续 TLP 的发送，因为这两个 TLP 的目的地址并不相同，发送条件也并不相同。

值得注意的是，在使用 PCI 总线强序模型时，不同种类的 TLP 间也可以乱序通过同一条 PCIe 链路，比如存储器写 TLP 可以超越存储器读请求 TLP 提前进行。而 PCIe 总线支持 Relaxed Ordering 模型之后，在 TLP 的传递过程中出现乱序种类更多，但是这些乱序仍然是有条件限制的。在 PCIe 总线规范中为了避免死锁，还规定了不同报文的传送数据规则，即 Ordering Rules。有关 PCIe 总线序的详细说明见第 11 章。

PCIe V2.1 总线规范引入了一种新的“序”模型，即 IDO（ID-Based Ordering）模型，IDO 模型与数据传送的数据流相关，是 PCIe V2.1 规范引入的序模型。有关 PCIe 总线的 Relaxed Ordering 和 IDO 模型的详细说明见第 11.4.2 节。

Attr 字段的第 0 位是“No Snoop Attribute”位。当该位为 0 时表示当前 TLP 所传送的数据在通过 FSB 时，需要与 Cache 保持一致，这种一致性由 FSB 通过总线监听自动完成而不需要软件干预；如果为 1，表示 FSB 并不会将 TLP 中的数据与 Cache 进行一致，在这种情况下，进行数据传送时，必须使用软件保证 Cache 的一致性。

在 PCI 总线中没有与这个“No Snoop Attribute”位对应的概念，因此一个 PCI 设备对存

存储器进行 DMA 操作时会进行 Cache 一致性操作^①。这种“自动的”Cache 一致性行为在某些特殊情况下并不能带来更高的效率。

当一个 PCIe 设备对存储器进行 DMA 读操作时，如果传送的数据非常大，比如 512MB，Cache 的一致性操作不但不会提高 DMA 写的效率，反而会降低。因为这个 DMA 读访问的数据在绝大多数情况下，并不会在 Cache 中命中，但是 FSB 依然需要使用 Snoop Phase 进行总线监听。而处理器在进行 Cache 一致性操作时仍然需要占用一定的时钟周期，即在 Snoop Phase 中占用的时钟周期，Snoop Phase 是 FSB 总线事务的一个阶段，如图 3-6 所示。

对于这类情况，一个较好的做法是，首先使用软件指令保证 Cache 与主存储器的一致性，并置“No Snoop Attribute”位为 1^②，然后再进行 DMA 读操作。同理使用这种方法对一段较大的数据区域进行 DMA 写时，也可以提高效率。

除此之外，当 PCIe 设备访问的存储器，不是“可 Cache 空间”时，也可以通过设置“No Snoop Attribute”位，避免 FSB 的 Cache 共享一致性操作，从而提高 FSB 的效率。“No Snoop Attribute”位是 PCIe 总线针对 PCI 总线的不足做出的重要改动。

6.1.4 通用 TLP 头中的其他字段

除了 Fmt 和 Type 字段外，通用 TLP 头还含有以下字段。

1. TH 位、TD 位和 EP 位

TH 位为 1 表示当前 TLP 中含有 TPH（TLP Processing Hint）信息，TPH 是 PCIe V2.1 总线规范引入的一个重要功能。TLP 的发送端可以使用 TPH 信息，通知接收端即将访问数据的特性，以便接收端合理地预读和管理数据，TPH 的详细介绍见第 6.3.6 节。

TD 位表示 TLP 中的 TLP Digest 是否有效，为 1 表示有效，为 0 表示无效。而 EP 位表示当前 TLP 中的数据是否有效，为 1 表示无效，为 0 表示有效。

2. AT 字段

AT 字段与 PCIe 总线的地址转换相关。在一些 PCIe 设备中设置了 ATC（Address Translation Cache）部件，这个部件的主要功能是进行地址转换。只有在支持 IOMMU 技术的处理器系统中，PCIe 设备才能使用该字段。

AT 字段可以用作存储器域与 PCI 总线域之间的地址转换，但是设置这个字段的主要目的是为了更方便多个虚拟主机共享同一个 PCIe 设备。对这个字段有兴趣的读者可以参考 Address Translation Services 规范，这个规范是 PCI 的 IO Virtualization 规范的重要组成部分。对虚拟化技术有兴趣的读者可以参考清华大学出版社的《系统虚拟化——原理与实现》，以获得基本的关于虚拟化的入门知识。该书主要针对处理器系统的虚拟化技术。而本书将在第 13.2.1 节详细介绍 AT 字段和 PCI 总线相关的虚拟化技术。

3. Length 字段

Length 字段用来描述 TLP 的有效负载（Data Payload）大小^③。PCIe 总线规范规定一个 TLP 的 Data Payload 的大小在 0~4096B 之间。PCIe 总线设置 Length 字段的目的是提高总

① PowerPC 处理器通过设置 Inbound 寄存器，也可以避免这个 Cache 一致性操作。

② FSB 收到这类 TLP 后，不进行 Cache 一致性操作。

③ 存储器读请求 TLP 没有 Data Payload 字段，此时该 TLP 使用 Length 字段表示需要读取多少数据。

线的传送效率。

当 PCI 设备在进行数据传送时，其目标设备并不知道实际的数据传送大小，这在一定程度上影响了 PCI 总线的数据传送效率。而在 PCIe 总线中，目标设备可以通过 Length 字段提前获知源设备需要发送或者请求的数据长度，从而合理地管理接收缓冲，并根据实际情况进行 Cache 一致性操作。

当 PCI 设备进行 DMA 写操作，将 PCI 设备中 4 KB 大小的数据传送到主存储器时，这个 PCI 设备的 DMA 控制器将存放传送的目的地址和传送大小，然后启动 DMA 写操作，将数据写入到主存储器。由于 PCI 总线是一条共享总线，因此传送 4 KB 大小的数据，可能会使用若干个 PCI 总线写事务才能完成^①，而每一个 PCI 总线写事务都不知道 DMA 控制器何时才能将数据传送完毕。

如果这些总线写事务还通过一系列 PCI 桥才能到达存储器，在这个路径上的每一个 PCI 桥也无法预知这个 DMA 操作何时才能结束，那么这种“不可预知”将导致 PCI 总线的带宽不能被充分利用，而且极易造成 PCI 桥数据缓冲的浪费。

而 PCIe 总线通过 TLP 的 Length 字段，可以有效避免 PCIe 链路带宽的浪费。值得注意的是，Length 字段以 DW 为单位，其最小单位为 1 个 DW。如果 PCIe 主设备传送的单位小于 1 个 DW 或者传送的数据并不以 DW 对界时，需要使用字节使能字段，即“DW BE”字段。有关“DW BE”字段的详细说明见第 6.3.1 节。

6.2 TLP 的路由

TLP 的路由是指 TLP 通过 Switch 或者 PCIe 桥片时采用哪条路径，最终到达 EP 或者 RC 的方法。PCIe 总线一共定义了三种路由方法，分别是基于地址（Address）的路由，基于 ID 的路由和隐式路由（Implicit）方式。

存储器和 I/O 读写请求 TLP 使用基于地址的路由方式，这种方式使用 TLP 中的 Address 字段进行路由选径，最终到达目的地。

而配置读写报文、“Vendor_Defined Messages”报文、Cpl 和 CplD 报文使用基于 ID 的路由方式，这种方式使用 PCI 总线号^②（Bus Number）进行路由选径。在 Switch 或者多端口 RC 的虚拟 PCI-to-PCI 桥配置空间中，包含如何使用 PCI 总线号进行路由选径的信息。

而隐式路由方式主要用于 Message 报文的传递。在 PCIe 总线中定义了一系列消息报文，包括“INTx Interrupt Signaling”，“Power Management Messages”和“Error Signal Messages”等报文。在这些报文中，除了“Vendor_Defined Messages”报文，其他所有消息报文都使用隐式路由方式，隐式路由方式是指从下游端口到上游端口进行数据传递的使用路由方式，或者用于 RC 向 EP 发出广播报文。

6.2.1 基于地址的路由

在 PCIe 总线中，存储器读写和 I/O 读写 TLP 使用基于地址的路由方式。PCIe 设备使用

① 当多个 PCI 设备共享一条 PCI 总线时，一个设备不会长时间占用 PCI 总线，这个设备在使用这条 PCI 总线一定的时间后，将让出 PCI 总线的使用权。

② PCIe 总线实际上使用 Transaction ID 进行 ID 路由，有关 Transaction ID 的详细说明见第 6.3.1 节。

的地址路由方式与 PCI 设备使用的地址路由方式类似。只是 PCIe 设备使用 TLP 进行数据传送，而 PCI 设备使用总线周期进行数据传送。使用地址路由方式进行数据传递的 TLP 格式如第 6.3.1 节的图 6-8 所示，在这类 TLP 中包含目的设备的地址。

当一个 TLP 进行数据传递时，可能会经过多级 Switch，最终到达目的地。Switch 将根据存储器读写和 I/O 读写请求 TLP 的目的地址将报文传递到合适的 Egress 端口上。如图 4-10 所示，在一个 Switch 中包含了多个虚拟 PCI-to-PCI 桥。在 Switch 中有几个端口，就包含几个虚拟 PCI-to-PCI 桥。

在虚拟 PCI-to-PCI 桥的配置寄存器空间中，包含一个桥片能够接收的物理地址范围。PCIe 总线通过这个物理地址范围实现基于地址的路由。这段配置寄存器如图 6-4 所示。当系统软件初始化 PCI 总线时，将合理地设置这些寄存器，之后当 TLP 通过这些 Switch 时将根据这些寄存器选择合适的路径。

Secondary Status	I/O Limit	I/O Base	1Ch
Memory Limit	Memory Base		20h
Prefetchable Memory Limit	Prefetchable Memory Base		24h
Prefetchable Base Upper 32 Bits			28h
Prefetchable Limit Upper 32 Bits			2Ch
I/O Limit Upper 16 Bits	I/O Base Upper 16 Bits		30h

图 6-4 与地址路由相关的 PCI 桥片配置寄存器

图 6-4 中的配置寄存器描述了该虚拟 PCI-to-PCI 桥下游 PCI 子树使用的三组空间范围，分别为 I/O、存储器和可预取的存储器空间，分别用 Base 和 Limit 两类寄存器描述，其中 Base 寄存器表示可访问空间的基地址，Limit 寄存器表示可访问空间的大小。TLP 使用基于地址的路由时，一定要通过查询这组寄存器之后，再决定传送路径。这组寄存器的使用方法与 PCI 总线中的 PCI 桥兼容。

其中 TLP “从上游端口发送到下游端口”与“从下游端口发送到上游端口”的路由过程略有不同，如图 6-5 所示。下文以 TLP1~TLP3 的发送过程对地址路由过程进行说明。TLP1~TLP3 的描述如下。

- TLP1 是一个存储器或者 I/O 请求 TLP，由 RC 发出，并通过一个 Switch 发向 EP1。存储器和 I/O 读写请求 TLP 使用这种地址路由方式。TLP1 将从 Switch 的上游端口传送到下游端口。
- TLP2 是一个存储器或者 I/O 请求 TLP，由 EP2 发出，并通过一个 Switch 发向 RC。当 PCIe 设备进行 DMA 读写操作时，将使用这种地址路由方式。TLP2 将从 Switch 的下游端口传送到上游端口。
- TLP3 是一个存储器或者 I/O 请求 TLP，由一个 EP2 发出，并通过一个 Switch 后发送到另外一个 EP。在 x86 处理器系统中，这种用法并不常见。但是在某些大规模处理器系统中，具有这种应用方式。此时 TLP3 将从 Switch 的下游端口传送到另外一个下游端口。

1. TLP1 的传送过程

当 TLP1 从 RC 发向 EP1 时，这个 TLP1 为 I/O 或者存储器报文，其中 TLP1 目的地址在 EP1 的 BAR 空间中。当处理器访问 EP 的 BAR 空间时，需要使用该类 TLP。值得注意的是

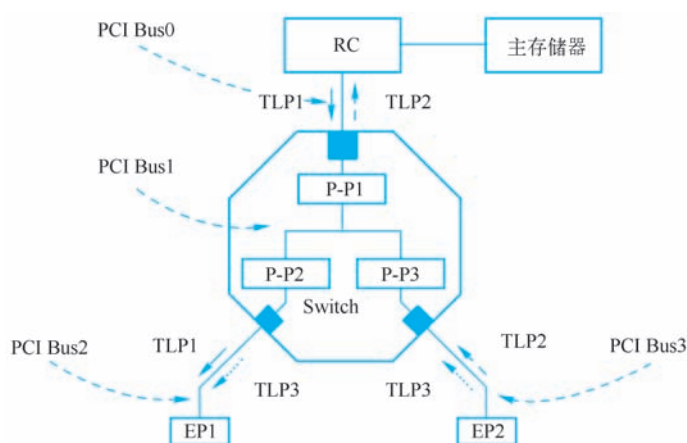


图 6-5 基于地址的路由寻径方式

这个数据报文在通过 RC 时需要进行地址转换。

TLP1 首先通过 PCI Bus0 发向 Switch，并通过 Switch 的 Upstream 端口到达 P-P1 桥片，P-P1 桥片首先根据配置寄存器中的 Limit 和 Base 寄存器决定是否接收 TLP1。如果 Switch 不接收 TLP1，则将该 TLP 作为不支持的请求（Unsupported Request）处理，此时如果 TLP1 需要回应报文，Switch 将发出完成报文，该报文的状态为 UR（Unsupported Request）。

如果 Switch 接收 TLP1，则表示 TLP1 所访问的地址在该 Switch 下游端口所连接的 EP 或者 Switch 中，此时 Switch 将 TLP1 从 PCI Bus0 推至 PCI Bus1 中，即穿越 P-P1 桥片。TLP1 到达 PCI Bus1 后将同时查找 P-P2 和 P-P3 桥片配置寄存器中的 Limit 和 Base 寄存器，决定是 P-P2 还是 P-P3 桥片接收 TLP1。本小节中的例子将使用 P-P2 桥片接收 TLP1，并将 TLP1 推至 PCI Bus2，而 PCI Bus2 上的 EP1 将接收 TLP1，完成整个地址路由。

2. TLP2 的传送过程

当 TLP2 从 EP2 发向 RC 时，一般来说该 TLP 将访问处理器系统的主存储器。此时 TLP2 首先将请求发至 P-P3 桥片，在 P-P3 桥片配置寄存器的 Limit 和 Base 寄存器中当然不会包含 TLP2 所访问的地址，此时 P-P3 桥片将 TLP2 推至 PCI Bus1。

TLP “从下游端口向上游端口”与“从 TLP 从上游端口向下游端口”进行传递时，桥片的处理机制有所不同，从上游端口向下游端口传递时，如果桥片配置寄存器的 Limit 和 Base 寄存器包含该 TLP 的访问地址时，桥片将接收此 TLP，否则不接收该 TLP。而从下游端口向上游端口传递时，如果桥片配置寄存器的 Limit 和 Base 寄存器不包含该 TLP 的访问地址时，桥片将接收该 TLP，并将其推至桥片的上游 PCI 总线。值得注意的是，这两种地址译码方式都属于 PCI 总线的正向译码。

当 TLP2 到达 PCI Bus1 时，首先检查在 PCI Bus1 总线上的 P-P2 桥片是否可以接收此 TLP，如果不能接收则 TLP2 通过 P-P1 桥片传递到 PCI Bus0，即到达 RC。

在 MPC8548 处理器中，到达 RC 的 TLP 首先通过 Inbound 寄存器进行地址转换，将 TLP 的 PCI 总线地址转换为处理器的地址，然后访问处理器中相应的存储器空间；对于 x86 处理器而言，MCH 也会完成 PCI 域地址空间到存储器域地址空间的转换，然后访问处理器中相应的存储器空间。

3. TLP3 的传递过程

TLP3 的传递方式与 TLP2 的传递方式有些类似，当 TLP3 传递到 PCI Bus1 时，P-P2 桥片将接收 TLP3，并将 TLP3 传递到 PCI Bus2 上的 EP1 中。由以上叙述可以发现，PCIe 总线中基于地址的路由方式与 PCI 总线上的基于地址的数据传递流程十分相近。TLP3 在 PCI 总线域上进行数据传递，因此不需要进行 PCI 总线域到存储器域的地址转换。

6.2.2 基于 ID 的路由

在 PCIe 总线中，基于 ID 的路由方式主要用于配置读写请求 TLP、Cpl 和 CplID 报文，此外 Vendor_Defined 消息报文也可以使用这种基于 ID 的路由方式。而在 PCI 总线中，只有配置读写周期才使用 ID 进行数据传递。

基于 ID 的路由方式与基于地址的路由方式有较大的不同，基于 ID 路由方式的 TLP 头格式也与基于地址路由方式的头格式不同，其报文格式如图 6-6 所示。

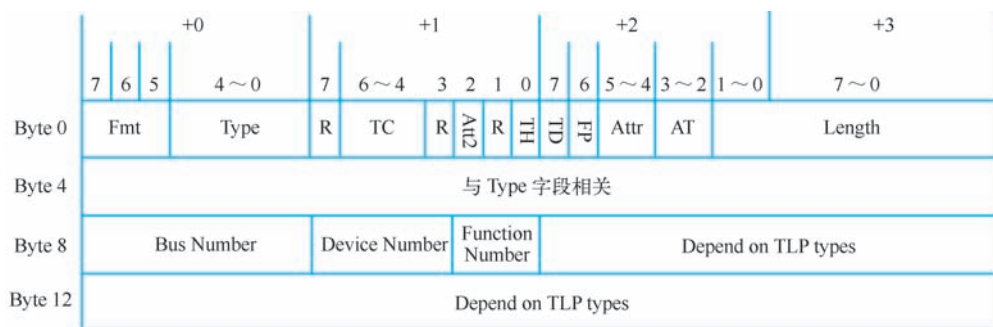


图 6-6 使用 ID 路由的 TLP 头格式

使用 ID 路由方式的 TLP 头，其 Byte8~Byte11 字段与基于地址路由的 TLP 不同。基于 ID 路由的 TLP，使用 Bus Number、Device Number 和 Function Number 进行路由寻址。从软件的角度来看，PCIe 总线与 PCI 总线兼容，只是在 PCIe 总线中，每一个 PCIe 设备使用唯一的 PCI 设备号，但是每一个设备仍然可以有多个子设备（Function）。

PCIe 总线规定，在一个 PCI 总线域空间中，最多只能有 256 条 PCI 总线，因此在一个 TLP 中，Bus Number 由 5 位组成；而在每一条总线中最多包含 32 个设备，因此 TLP 中的 Device Number 由 5 位^①组成；而每一个设备中最多包含 8 个功能，因此一个 TLP 的 Function Number 由 3 位组成。

配置读写请求 TLP 是使用“基于 ID 路由”的一组重要报文，其主要作用是读写 PCIe 总线的 EP、Switch 及 PCIe 桥片的配置寄存器，以完成 PCIe 总线的配置。在处理器系统上电之后需要进行 PCI 总线系统的枚举，为 PCI 总线分配总线号，并设置 Switch、PCIe 桥片或者 EP 的配置寄存器，如 Limit 寄存器组、Base 寄存器组、BAR 寄存器、Subordinate Bus Number、Secondary Bus Number 和 Primary Bus Number 等一系列配置寄存器。

在上文中我们简单介绍了 Limit 寄存器组和 Base 寄存器组的用法，下文将重点描述 Sub-

① PCIe 链路采用端到端的通信方式，每一个链路只能挂接一个设备，因此在多数情况下，使用 3 位描述 Device Number 是多余的，因此 PCIe 总线提出了 ARI 格式，该格式的详细描述见第 6.3.1 节。

ordinate Bus Number、Secondary Bus Number 和 Primary Bus Number 寄存器。Subordinate Bus Number、Secondary Bus Number 和 Primary Bus Number 寄存器在 Type 01h 配置寄存器中，用来描述 PCI-to-PCI 桥片的上游及下游总线号。这段寄存器在 PCI 配置寄存器中的位置如图 6-7 所示。



图 6-7 与 ID 路由相关的 PCI 桥片配置寄存器

与 PCI 总线中的桥片类似，Primary Bus Number 记录 PCI-to-PCI 桥上游的 PCI 总线号，Secondary Bus Number 记录 PCI-to-PCI 桥下游的第一个 PCI 总线号，而 Subordinate Bus Number 记录 PCI-to-PCI 桥下游的最后一个 PCI 总线号。

如图 6-5 所示，P-P1 桥片的 Primary Bus Number 为 0，Secondary Bus Number 为 1，而 Subordinate Bus Number 为 3。这些总线号，在处理器系统对 PCI 总线进行枚举时由系统初始化程序设置，从系统初始化程序的角度来看，PCIe 总线与 PCI 总线基本兼容，只是 PCIe 总线对配置空间进行了一些扩展。

在如表 6-2 所示中，RC 可以使用 Type 00h 和 Type 01h 读写请求 TLP，对 PCIe 设备的配置寄存器进行读写访问，配置读写请求 TLP 只能由 RC 发出，配置读写请求 TLP 使用基于 ID 的路由方式。

在如图 6-5 所示的例子中，RC 首先使用 Type 00h 配置请求 TLP 访问在 PCI Bus0 总线上的设备，PCI Bus0 上的所有设备，包括桥片都要监听 PCI Bus 0 上的配置请求，在本例中只有 Switch 挂接在 PCI Bus0 上，实际上是 Switch 的上游端口与 PCI Bus0 直接相连。因此 Switch 的上游端口将接收 RC 发出的 Type 00h 配置请求 TLP，之后 Switch 将向 RC 发出完成报文，结束配置请求。与 PCI 总线相同，PCIe 总线的 Type 00h 类型配置请求 TLP 不能够穿越桥片，在图 6-5 中这类请求只能访问 Switch 上游端口的配置空间。

PCI 总线是基于共享总线的数据传送方式，在一条 PCI 总线上可以连接多个 PCI Agent 设备，其中每一个 PCI Agent 都提供了一个 IDSEL#信号，这个信号与 PCI-to-PCI 桥片或者 HOST 主桥的地址线直接相连，PCI 总线根据与 IDSEL#信号与地址线的连接关系决定相应设备的 Device Number。

这与 PCIe 总线的使用方法不同，PCIe 总线使用“端对端”的连接方式，PCIe 链路只能连接一个下游设备，而这个下游设备的 Device Number 只能为 0。而只有在 Switch 的虚拟 PCI 总线上可以连接多个 Device Number 不同的端口。

当一个虚拟 PCI 总线上挂接 PCI-to-PCI 桥时，系统配置软件将使用 Type 01h 配置请求 TLP 访问 PCI-PCI 桥下游的 PCI 设备。如图 6-5 所示，RC 可以通过 Type 01h 配置请求 TLP 访问 P-P2 桥片、P-P3 桥片，EP1 和 EP2。

当 RC 使用 Type 01h 配置请求 TLP，直接访问 P-P1 桥的下游设备时，首先需要检查该 TLP 的 Bus Number 是否为 1，如果为 1 表示该 TLP 的访问目标在 PCI Bus 1 总线上，此时 PCI-to-PCI 桥将这个 Type 01h 类型的 TLP 转换为 Type 00h 类型的 TLP，然后推至 PCI Bus 1 总线，并访问其下的设备。

如果该 TLP 的 Bus Number 在 P-P1 桥片的 Secondary Bus Number 和 Subordinate Bus Num-

ber 寄存器之间，则 P-P1 桥片将该 Type 01h 类型的 TLP 直接透传到 PCI Bus 1 上，并不改变该 TLP 的类型，之后 Type 01h 类型的 TLP 将继续检查 P-P2 和 P-P3 桥片的配置空间，决定由 P-P2 还是 P-P3 接收该 TLP。如果 TLP 的 PCI Bus Number 为 2 时，P-P2 桥片将接收该 TLP，并将该 Type 01h 类型 TLP 转换为 Type 00h 类型的 TLP，然后发送给 EP1，并由 EP1 处理该 TLP。

上文简要讲述了配置请求 TLP 使用 ID 路由方式从上游端口向下游端口的传递规则，但是 Vendor_Defined 消息报文和 Cpl 和 CplD 报文还可能从下游端口向上游端口进行传递。此时 PCIe 总线处理方法略有不同。下文仍以图 6-5 为例说明这种情况。

当一个 TLP 从 EP2 传送到 EP1 或者 RC 时，首先检查 P-P3 桥片的配置空间，P-P3 桥片发现该 TLP 不是发向自己时，将该 TLP 推至上游总线，即 PCI Bus1。如果 PCI Bus1 上 P-P1 桥片没有认领该 TLP，该 TLP 将继续向 P-P2 桥片传递，并由这个桥片将 TLP 转发给合适的 EP；如果 P-P1 桥片认领该 TLP，该 TLP 将继续向上游总线传递，直至 RC。

由以上描述可以发现，PCIe 总线使用的基于 ID 的路由方式与 PCI 总线中配置读写总线事务通过 PCI 桥的方法较为类似。

6.2.3 隐式路由

PCIe 总线规定消息请求报文使用隐式路由方式。在 PCIe 总线中，有许多消息是直接发向 RC 或者来自 RC 的广播报文，这些报文不使用地址或者 ID 进行路由，而是使用 Msg 和 MsgD 报文的 Route 字段进行路由，这种路由方式称为隐式路由。

PCIe 总线定义了一些用于中断请求、错误状态处理、锁定总线事务、热插拔信号处理和“Vendor_Defined Messages”消息报文。这些消息报文需要使用隐式路由方式进行传递。消息报文的 Route 字段的含义如表 6-4 所示。

表 6-4 Route[4:0] 字段

Route[4:0]	描 述
000	路由到 RC
001	使用地址路由
010	使用 ID 路由
011	来自 RC 的广播报文
100	本地消息，在接收端结束（Legacy 中断消息使用这种报文格式，传递来自 PCI 总线的中断报文）
101	用于 PCIe 电源管理（PME_TO_Ack 报文使用）
110~111	保留

使用隐式路由方式的 TLP，其 Route 字段为“000”，“011”，“100”或者“101”。当一个报文使用隐式路由向 EP 发送时，EP 将对 Route 字段进行检查，如果这个报文是“来自 RC 的广播报文”，或者是“本地报文”，EP 将接收此报文。

如果 Switch 收到一条使用隐式路由的 TLP 时，将根据报文 Route 字段的值而分别处理。如果 Switch 的上游端口接收了一条来自 RC 的广播消息，则将该报文发向所有的下游端口；如果 Switch 接收了一条来自下游端口发向 RC 的消息报文时，Switch 将此报文直接转发

到上游端口，直至 RC；如果 Switch 接收了一条使用隐式路由方式的本地消息报文，则 Switch 接收并终结此报文，不再上传或下推。

如果 RC 收到一个使用隐式路由的 TLP 时，将根据报文 Route 字段而分别处理这些 TLP。如果该 Route 字段为 0b000 和 0b101，RC 将接收该 TLP，并作相应的处理；如果为 0b100，RC 将接收该 TLP，并结束该 TLP 报文的传递。

6.3 存储器、I/O 和配置读写请求 TLP

本节讲述 PCIe 总线定义的各类 TLP，并详细介绍这些 TLP 的格式。在这些 TLP 中，有些格式对于初学者来说较难理解。为此本书将在第 12 章中结合一个设计实例，进一步描述这些 TLP 格式。

但是在阅读第 12 章的内容之前，读者需要建立 PCIe 总线中与 TLP 相关的一些基本概念，特别是存储器读写相关的报文格式。在 PCIe 总线中，存储器读写，I/O 读写和配置读写请求 TLP 由以下几类报文组成。

(1) 存储器读请求 TLP 和读完成 TLP

当 PCIe 主设备，RC 或者 EP，访问目标设备的存储器空间时，使用 Non-Posted 总线事务向目标设备发出存储器读请求 TLP，目标设备收到这个存储器读请求 TLP 后，使用存储器读完成 TLP，主动向主设备传递数据。当主设备收到目标设备的存储器读完成 TLP 后，将完成一次 DMA 读操作。

(2) 存储器写请求 TLP

在 PCIe 总线中，存储器写使用 Posted 总线事务。PCIe 主设备仅使用存储器写请求 TLP 即可完成 DMA 写操作，主设备不需要目标设备的回应报文。

(3) 原子操作请求和完成报文

原子操作由 PCIe V2.1 总线规范引入，一个完整的原子操作由原子操作请求和原子操作完成报文组成。原子操作的使用方法与其他 Non-Posted 总线事务类似，首先 PCIe 主设备向目标设备发送原子操作请求，之后目标设备向主设备发送原子操作完成报文，结束一次原子操作。有关原子操作的详细说明见第 6.3.5 节。

(4) I/O 读写请求 TLP 和读写完成 TLP

在 PCIe 总线中，I/O 读写操作使用 Non-Posted 总线事务，I/O 读写 TLP 都需要完成报文作为回应。只是在 I/O 写请求的完成报文中不需要“带数据”，而仅含有 I/O 写请求是否成功的状态信息。

(5) 配置读写请求 TLP 和配置读写完成 TLP

从总线事务的角度上看，配置读写请求操作的过程与 I/O 读写操作的过程类似。配置读写请求 TLP 都需要配置读写完成作为应答，从而完成一个完成的配置读写操作。

(6) 消息报文

与 PCI 总线相比，PCIe 总线增加了消息请求事务。PCIe 总线使用基于报文的数据传送模式，所有总线事务都是通过报文实现的，PCIe 总线取消了一些在 PCI 总线中存在的边带信号。在 PCIe 总线中，一些由 PCI 总线的边带信号完成的工作，如中断请求和电源管理等，在 PCIe 总线中由消息请求报文实现。

6.3.1 存储器读写请求 TLP

存储器读写请求 TLP 头格式如图 6-8 所示。

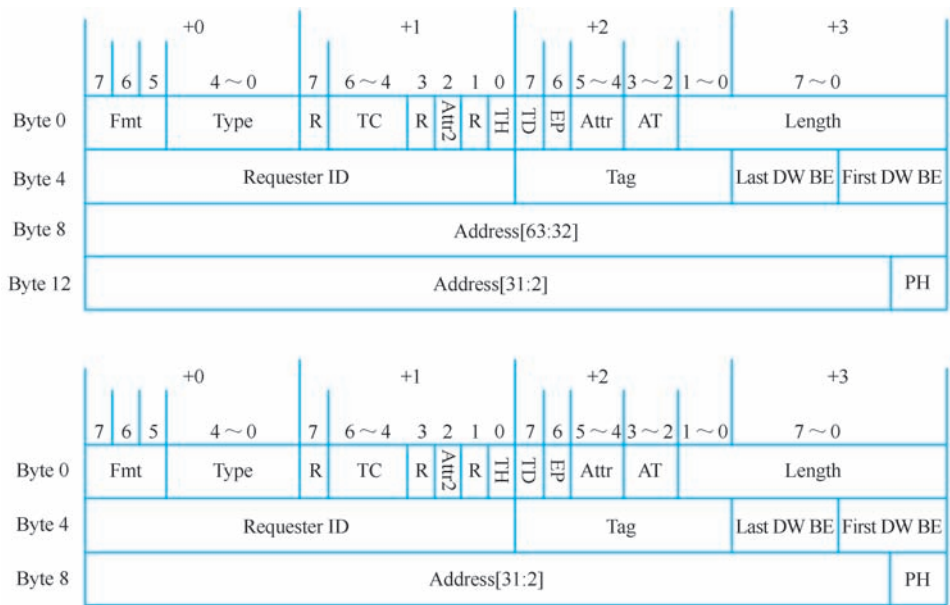


图 6-8 存储器和 I/O 读写请求 TLP 头格式

在 PCIe 总线中，存储器写请求 TLP 使用 Posted 数据传送方式。而其他与存储器和 I/O 相关的报文都使用 Split 方式进行数据传送，这些请求报文需要完成报文，通知发送端之前的数据请求报文已经处理完毕，有关完成报文的详细说明见第 6.3.2 节。

存储器读写请求 TLP 使用地址路由方式进行数据传递，在这类 TLP 头中包含 Address 字段，Address 字段具有两种地址格式，分别是 32 位和 64 位地址。在存储器读写和 I/O 读写请求的第 3 和第 4 个双字中，存放 TLP 的 32 或者 64 位地址。存储器、I/O 和原子操作读写请求使用的 TLP 头较为类似。本节仅介绍存储器、I/O 读写使用的 TLP 头，而在第 6.3.5 节详细介绍原子操作。

1. Length 字段

在存储器读请求 TLP 中并不包含 Data Payload，在该报文中，Length 字段表示需要从目标设备数据区域读取的数据长度；而在存储器写 TLP 中，Length 字段表示当前报文的 Data Payload 长度。

Length 字段的最小单位为 DW。当该字段为 n 时，表示需要获得的数据长度或者当前报文的数据长度为 n 个 DW，其中 $0 \leq n \leq 0x3FF$ 。值得注意的是，当 n 等于 0 时，表示数据长度为 1024 个 DW。

2. DW BE 字段

PCIe 总线以字节为基本单位进行数据传递，但是 Length 字段以 DW 为最小单位。为此 TLP 使用 Last DW BE 和 First DW BE 这两个字段进行字节使能，使得在一个 TLP 中，有效数据以字节为单位。

这两个 DW BE 字段各由 4 位组成，其中 Last DW BE 字段的每一位对应数据 Payload 最后一个双字的字节使能位；而 First DW BE 字段的每一位对应数据 Payload 第一个双字的字节使能位。其对应关系如表 6-5 所示。

表 6-5 First 和 Last DW BE 字段

Last DW BE	第 3 位	为 1 表示数据 Payload 的最后一个双字的字节 3 有效
	第 2 位	为 1 表示数据 Payload 的最后一个双字的字节 2 有效
	第 1 位	为 1 表示数据 Payload 的最后一个双字的字节 1 有效
	第 0 位	为 1 表示数据 Payload 的最后一个双字的字节 0 有效
First DW BE	第 3 位	为 1 表示数据 Payload 的第一个双字的字节 3 有效
	第 2 位	为 1 表示数据 Payload 的第一个双字的字节 2 有效
	第 1 位	为 1 表示数据 Payload 的第一个双字的字节 1 有效
	第 0 位	为 1 表示数据 Payload 的第一个双字的字节 0 有效

Last DW BE 和 First DW BE 这两个字段的使用规则如下。

- 如果传送的数据长度在一个对界的双字（DW）之内，则 Last DW BE 字段为 0b0000，而 First DW BE 的对应位置 1；如果数据长度超过 1DW，Last DW BE 字段一定不能为 0b0000。PCIe 总线使用 Last DW BE 字段为 0b0000 表示所传送的数据在一个对界的 DW 之内。
- 如果传送的数据长度超过 1DW，则 First DW BE 字段至少有一个位使能。不能出现 First DW BE 为 0b0000 的情况。
- 如果传送的数据长度大于等于 3DW，则在 First DW BE 和 Last DW BE 字段中不能出现不连续的置 1 位。
- 如果传送的数据长度在 1DW 之内时，在 First DW BE 字段中允许有不连续的置 1 位。此时 PCIe 总线允许在 TLP 中传送 1 个 DW 的第 1, 3 字节或者第 0, 2 字节。
- 如果传送的数据长度在 2DW 之内时，则 First DW BE 字段和 Last DW BE 字段允许有不连续的置 1 位。

值得注意的是，PCIe 总线支持一种特殊的读操作，即“Zero-Length”读请求。此时 Length 字段的长度为 1DW，而 First DW BE 字段和 Last DW BE 字段都为 0b0000，即所有字节都不使能。此时与这个存储器读请求 TLP 对应的读完成 TLP 中不包含有效数据。再次提醒读者注意“Zero-Length”读请求使用的 Length 字段为 1，而不为 0，为 0 表示需要获得的数据长度为 1024 个 DW。

“Zero-Length”读请求的引入是为了实现“读刷新”操作，该操作的主要目的是为了确保之前使用 Posted 方式所传送的数据，到达最终的目的地，与“Zero-Length”读对应的读完成报文中不含有负载，从而提高了 PCIe 链路的利用率。

在 PCIe 总线中，使用 Posted 方式进行存储器写时，目标设备不需要向主设备发送回应报文，因此主设备并不知道这个存储器写是否已经到达目的地。而主设备可以使用“读刷新”操作，向目标设备进行读操作来保证存储器写最终到达目的地。有关“读刷新”的详细说明及实现原理见第 11 章。

在 PCIe 总线中，标准的存储器读请求也可以完成同样的刷新操作。但是“Zero-Length”读请求与这种读请求相比，其完成报文不需要“Data Payload”，因此在一定程度上提高了 PCIe 总线的效率。如果一个存储器读请求 TLP 报文的 TH 位为 1 时，DW BE 字段将被重新定义为 ST[7:0] 字段，有关 ST 字段的详细说明见第 6.3.6 节。

3. Requester ID 字段

Requester ID 字段包含“生成这个 TLP 报文”的 PCIe 设备的总线号（Bus Number）、设备号（Device Number）和功能号（Function Number），其格式如图 6-9 所示。对于存储器写请求 TLP，Requester ID 字段并不是必需的，因为目标设备收到存储器写请求 TLP 后，不需要完成报文作为应答，因此 Requester ID 字段对于存储器写请求 TLP 并没有实际意义。

但是 PCIe 总线规范并没有明确说明存储器写请求 TLP 究竟需不需要 Requester ID 字段，为此 IC 设计者依然需要将存储器写 TLP 的 Requester ID 字段置为有效。值得注意的是，如果一个存储器写请求 TLP 报文的 TH 位为 1 时，Tag 字段将被重新定义为 ST[7:0] 字段，有关 ST 字段的详细说明见第 6.3.6 节。

对于 Non-Posted 数据请求，目标设备需要使用完成报文作为回应。在这个完成报文中，需要使用源设备的 Requester ID 字段。因此在 Non-Posted 数据请求 TLP 中，如存储器读请求、I/O 和配置读写请求 TLP，必须使用 Requester ID 字段。

存储器，I/O 读请求 TLP 中含有 Requester ID 和 Tag 字段。在 PCIe 总线中 Requester ID 和 Tag 字段合称为 Transaction ID，Transaction ID 字段的格式如图 6-9 所示。存储器读，I/O 和配置读写请求 TLP 使用 Transaction 字段的主要目的是使接收端通过分析报文的 Transaction ID，确认完成报文的目的地。

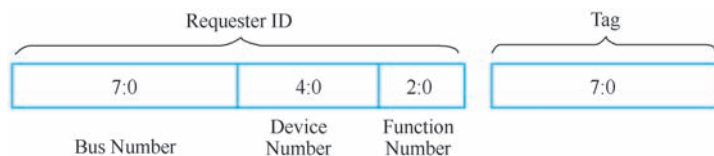


图 6-9 Transaction ID 的格式

在 PCIe 总线中，所有 Non-Posted 数据请求都需要完成报文作为应答，才能结束一次完整的数据传递。一个源设备在发送 Non-Posted 数据请求之后，如果并没有收到目标设备回送的完成报文，TLP 报文的发送端需要保存这个 Non-Posted 数据请求，此时该设备使用的 Transaction ID（Tag 字段）不能被再次使用，直到一次数据传送结束，即数据发送端收齐与该 TLP 对应的所有完成报文。

在同一个时间段内，PCIe 设备发出的每一个 Non-Posted 数据请求 TLP，其 Transaction ID 必须是唯一的。即在同一时间段内，在当前 PCI 总线域中不能存在两个或者两个以上的存储器读请求 TLP，其 Transaction ID 完全相同。

源设备发送 Non-Posted 数据请求后，在没有获得全部完成报文之前，不能释放这个 Transaction ID 占用的资源。在同一个 PCIe 设备发送的 TLP 中，其 Requester ID 字段是相同的，因此 PCIe 设备的设计者所能管理的资源是 Tag 字段。PCIe 设备的设计者需要合理地管理 Tag 资源，以保证数据传送的正确性。

PCIe 设备在发送 Non-Posted 数据请求时，需要暂存这些 Non-Posted 数据请求。其中 Tag

字段的长度决定了发送端能够暂存多少个同类型的 TLP，如果 Tag 字段长度为 5，发送端能够暂存 32 个报文；如果 PCIe 设备使能了 Extended Tag 位（该位的详细描述见第 4.3.2 节），Tag 字段可以由 8 位组成，此时发送端能够暂存 256 个报文。

通过 Tag 字段的长度，可以发现每个 PCIe 设备最多可以暂存 256 个同类型的 Non-Posted 报文。但是在多数情况下，一个 PCIe 设备可能只包含 1 个 Function。因此 PCIe 设备还可以使用 Function 号扩展 Tag 字段，从而扩展“暂存 TLP 报文”的数目。

PCIe 设备在 PCI Express Capability 结构的 Device Control 寄存器中，设置了一个 Phantom Functions Enable 位，该位的详细说明见第 4.3.2 节。当一个 PCIe 设备仅支持一个 Function 时，Phantom Functions Enable 位可以被设置为 1，此时 PCIe 设备可以使用 Requester ID 的 Function Number 字段对 Tag 字段进一步扩展，此时一个 PCIe 设备最多可以支持 2048 个同类型的数据请求。

由以上分析可以发现，一个 PCIe 设备最多可以支持 2048 个存储器读数据请求，基本上可以满足绝大多数需要。但是在某些特殊应用场合，PCIe 设备即使可以暂存 2048 个存储器读请求，也并不足够。

与 PCI 总线相比，PCIe 总线的数据传送延时较长，而为了弥补这个传送延时，PCIe 设备通常使用流水线技术。此时 PCIe 设备必须能够连续发送多个存储器读请求报文，随后 RC 也将连续回送多个存储器读完成报文，在 PCIe 设备的实现中，需要保证能够源源不断地从 RC 接收这些报文，以充分利用报文接收流水线，有关这部分内容详见第 12.4.3 节。

PCIe V2.1 总线规范还提出了另一种 Requester ID 格式，即 ARI (Alternative Routing-ID Interpretation) 格式，除了 Requester ID 外，在完成报文中使用的 Completer ID 也可以使用这种格式。ARI 格式将 ID 号分为两个字段，分别为 Bus 号和 Function 号，而不使用 Device 号，ARI 格式如图 6-10 所示。

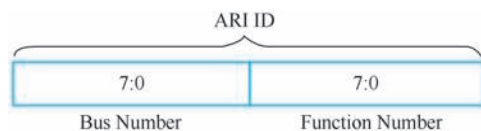


图 6-10 ARI 格式

PCIe 总线引入 ARI 格式的依据是在一个 PCIe 链路上仅可能存在一个 PCIe 设备，因而其 Device 号一定为 0。在多数 PCIe 设备中，Requester ID 和 Completion ID 包含的 Device 号是没有意义的。使用 ARI 格式时，一个 PCIe 设备最多可以支持 256 个 Function，而传统的 PCIe 设备最多只能支持 8 个 Function。

4. I/O 读写请求 TLP 的规则

I/O 读写请求与存储器读写请求 TLP 的格式基本类似，只是 I/O 读写请求 TLP 只能使用 32 位地址模式和基于地址的路由方式，而且 I/O 读写请求 TLP 只能使用 Non-Posted 方式进行传递。PCIe 总线并不建议 PCIe 设备支持 I/O 地址空间，但是 Switch 和 RC 需要具备接收和发送 I/O 请求报文的能力，因为许多老的 PCI 设备依然使用 I/O 地址空间，这些 PCI 设备可以通过 PCIe 桥连接到 PCIe 总线中。因此虽然支持 I/O 读写请求的 PCIe 设备极少，但是在 PCIe 体系结构中，依然需要支持 PCI 总线域的 I/O 地址空间。

与存储器读写请求 TLP 不同，I/O 读写请求 TLP 的某些字段必须为以下值。

- TC[2:0]必须为 0，I/O 请求报文使用的 TC 标签只能为 0。
- TH 和 Attr2 位保留，而 Attr[1:0]必须为 “0b00”，这表示 I/O 请求报文必须使用 PCI 总线的强序数据传送模式，而且在传送过程中，硬件保证其传送的数据与 Cache 保持一致，实际上 I/O 地址空间都是不可 Cache 的。
- AT[1:0]必须为 “0b00”，表示不支持地址转换，因此在虚拟化技术中，并不处理 PCI 总线域中的 I/O 空间。
- Length[9:0]为 “0b00 0000 0001”，表示 I/O 读写请求 TLP 最大的数据 Payload 为 1DW，该类 TLP 不支持突发传送。
- Last DW[3:0]为 “0b0000”。

6.3.2 完成报文

PCIe 总线支持 Split 传送方式，目标设备使用完成报文向源设备主动发送数据。完成报文使用 ID 路由方式，由 TLP Prefix、报文头和 Data Payload 组成，但是某些完成报文可以不含有 Data Payload，如 I/O 或者配置写完成和 Zero-Length 读完成报文。在 PCIe 总线中，有以下几类数据请求需要收到完成报文之后，才能完成整个数据传送过程，完成报文格式如图 6-11 所示。

- 所有的数据读请求，包括存储器、I/O 读请求、配置读请求和原子操作请求。当一个 PCIe 设备发出这些数据请求报文后，必须收到目标设备的完成报文后，才能结束一次数据传送。这一类完成报文必须包含 Data Payload。
- 所有的 Non-Posted 数据写请求，包括 I/O 和配置写请求。当一个 PCIe 设备发出这些数据请求报文后，必须收到目标设备的完成报文后，才能结束数据传送。但是这一类完成报文不包含数据，仅包含应答信息。
- 与 ATS 机制相关的一些报文，详见第 13.2 节。

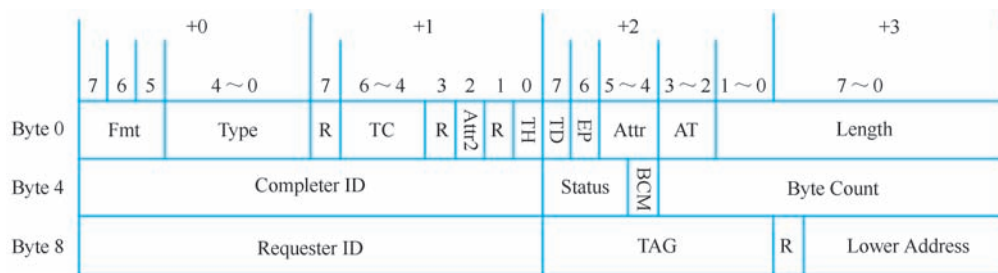


图 6-11 完成报文头格式

完成报文“Byte 0”中的大部分字段与“存储器、I/O、配置请求报文”的对应字段的含义相同。完成报文一次最多能够传送的报文大小不能超过 Max_Payload_Size 参数。在多数处理器中，完成报文中包含的数据在一个 Cache 行之内，完成报文使用 RCB 参数来处理数据对界，RCB 参数的大小与处理器系统的 Cache 行长度和 DDR-SDRAM 的一次突发传送长度相关，这些参数的详细描述见第 6.4.3 节。在 x86 和 PowerPC 处理器中，一个存储器读完成报文一般不超过 RCB 参数。

1. Requester ID 和 TAG 字段

完成报文使用 ID 路由方式，ID 路由方式详见第 6.2.2 节。完成报文头的长度为 3DW，完成报文头中包含 Transaction ID 信息，由 Requester ID 和 Tag 字段组成，这个 ID 必须与源设备发送的数据请求报文的 Transaction ID 对应，完成报文使用 Transaction ID 进行 ID 路由，并将数据发送给源设备。

当 PCIe 设备收到存储器读、I/O 读写或者配置读写请求 TLP 时，需要首先保存这些报文的 Transaction ID，之后当该设备准备好完成报文后，将完成报文的 Requester ID 和 Tag 字段赋值为之前保存的 Transaction ID 字段。

2. Completer ID 字段

Completer ID 字段的含义与 Requester ID 字段较为相似，只是该字段存放“发送完成报文”的 PCIe 设备的 ID 号。PCIe 设备进行数据请求时需要在 TLP 字段中包含 Requester ID 字段；而使用完成报文结束数据请求时，需要提供 Completer ID 字段。

3. Status 字段

Status 字段保存当前完成报文的完成状态，表示当前 TLP 是正确地将数据传递给数据请求端；还是在数据传递过程中出现错误；或者要求数据请求方进行重试。PCIe 总线规定了几类完成状态，如表 6-6 所示。

表 6-6 Status 字段

Status[2:0]	描 述
0b000	SC (Successful Completion)，正常结束
0b001	UR (Unsupported Request)，不支持的数据请求
0b010	CRS (Configuration Request Retry Status)，要求数据请求方进行重试。当 RC 对一个 PCIe 目标设备发起配置请求时，如果该目标设备没有准备好，可以向 RC 发出 CRS 完成报文，当 RC 收到这类报文时，不能结束本次配置请求，必须择时重新发送配置请求
0b100	CA (Completion Abort)，数据夭折。表示目标设备无法完成本次数据请求
其他	保留

4. BCM 位与 Byte Count 字段

BCM (Byte Count Modified) 字段由 PCI-X 设备设置。PCI-X 设备也支持 Split Transaction 传送方式，当 PCI-X 设备进行存储器读请求时，目标设备不一定一次就能将所有数据传递给源设备。此时目标设备在进行第一次数据传送时，需要设置 Byte Count 字段和 BCM 位。

BCM 位表示 Byte Count 字段是否被更改，该位仅对 PCI-X 设备有效，而 PCIe 设备不能操纵 BCM 位，只有 PCI-X 设备或者 PCIe-to-PCI-X 桥可以改变该位。本节对此位不做进一步介绍，对此位感兴趣的读者可以参考 PCI-X Addendum to the PCI Local Bus Specification, Revision 2.0。

Byte Count 字段记录源设备还需要从目标设备中获得多少字节的数据就能完成全部数据传递，当前 TLP 中的有效负载也被 Byte Count 字段统计在内。该字段由 12 位组成。该字段为 0b0000-0000-0001 表示还剩一个字节，为 0b1111-1111-1111 表示还剩 4095 个字节，而为

0b0000-0000-0000 表示还剩 4096 个字节。除了存储器读请求的完成报文外，大多数完成报文的 Byte Count 字段为 4。

如一个源设备向目标设备发送一个“读取 128B 的存储器读请求 TLP”，而目标设备收到这个读请求 TLP 后，可能使用两个存储器读完成 TLP 传递数据。其中第 1 个存储器读完成 TLP 的有效数据为 64B，而 Byte Count 字段为 128；第 2 个存储器读完成 TLP 中的有效数据为 64B，而 Byte Count 字段也为 64。当数据请求端接收完毕第 1 个存储器读完成 TLP 后，发现还有 64B 的数据没有接收完毕，此时必须等待下一个存储器读完成 TLP。等到数据请求端收齐所有数据后，才能结束整个存储器读请求。

目标设备发出的第 2 个读完成 TLP 中的有效数据为 64B，而 Byte Count 字段为 64，当数据请求端接收完毕这个读完成 TLP 后，将完成一个完整的存储器读过程，从而可以释放这个存储器读过程使用的 Tag 资源。

存储器读请求的完成报文的拆分方式较为复杂，Byte Count 字段的设置也相对较为复杂。在第 12 章将结合一个实例讲述该字段的使用方法。

5. Lower Address 字段

如果当前完成报文为存储器读完成 TLP，该字段存放在存储器读完成 TLP 中第一个数据所对应地址的最低位。值得注意的是，在完成报文中，并不存在 First DW BE 和 Last DW BE 字段，因此接收端必须使用存储器读完成 TLP 的 Low Address 字段，识别一个 TLP 中包含数据的起始地址。第 12.2.2 节将详细介绍该字段的作用。

6.3.3 配置读写请求 TLP

配置读写请求 TLP 由 RC 发起，用来访问 PCIe 设备的配置空间。配置请求报文使用基于 ID 的路由方式。PCIe 总线也支持两种配置请求报文，分别为 Type 00h 和 Type 01h 配置请求。配置请求 TLP 的格式如图 6-12 所示。

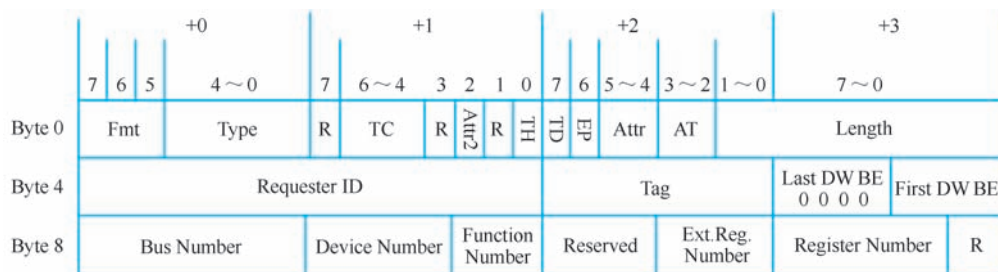


图 6-12 配置请求 TLP 的格式

配置请求 TLP 的第 4~7 字节与存储器请求 TLP 类似。而第 8~11 字节的 Bus、Device 和 Function Number 中存放该 TLP 访问的目标设备的相应的号码，而 Ext Register 和 Register Number 存放寄存器号。配置请求报文的其它字段必须为以下值。

- TC[2:0] 必须为 0，I/O 请求报文的传送类型（TC）只能为 0。
- TH 位为保留位；Attr2 位为保留，而 Attr[1:0] 必须为“00b”，这表示 I/O 请求报文使用 PCI 总线的强序数据传送模式；AT[1:0] 必须为“0b00”，表示不进行地址转换。

- Length[9:0] 为 “0b00 0000 0001”，表示配置读写请求最大 Payload 为 1DW。
- Last DW BE 字段为 “0b0000”。而 First DW BE 字段根据配置读写请求的大小设置。

6.3.4 消息请求报文

在 PCIe 总线中，多数消息报文使用隐式路由方式，其格式如图 6-13 所示。其中 Byte 0 字段为通用 TLP 头，而 Byte 4 的第 3 字节中存放 Message Code 字段。

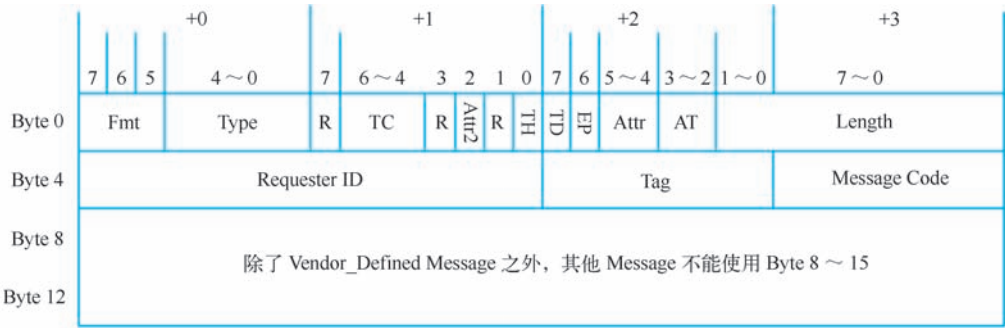


图 6-13 Message 请求 TLP 头格式

PCIe 总线规定了以下几类消息报文。

- INTx 中断消息报文（INTx Interrupt Signaling）。
- 电源管理消息报文（Power Management）。
- 错误消息报文（Error Signaling）。
- 锁定事务消息报文（Locked Transaction Support）。
- 插槽电源限制消息报文（Slot Power Limit Support）。
- Vendor-Defined Messages。

本节将重点讲述 INTx 中断和错误信息相关的消息报文，请读者阅读 PCIe 总线规范了解其他消息报文。

1. INTx 中断消息报文

PCIe 总线推荐设备使用 MSI 或者 MSI-X 机制提交中断请求，但是 MSI 中断机制并不是由 PCIe 总线首先提出的，在 PCI 总线中就已经存在这种中断请求机制。

在 PCI 总线中，虽然提出了 MSI 中断机制，但是几乎没有 PCI 设备使用这种机制进行中断请求。MSI 中断机制是一种基于存储器写的中断请求机制，而 PCI 设备提交 MSI 中断请求，将占用 PCI 总线的带宽，因此多数 PCI 设备使用 INTx 信号进行中断请求。

在 PCIe 总线中，PCIe 设备可以使用 Legacy 中断方式提交中断请求，此时需要使用 INTx 中断消息报文向 RC 通知中断事件。除此之外在 PCIe 体系结构中仍然存在 PCI 设备，这些设备可能使用 INTx 信号提交中断请求。

例如在 PCIe 桥片上挂接的 PCI 设备可能并不支持 MSI 中断机制，因此需要使用 INTx 中断信号提交中断请求，此时 PCIe 桥需要将 INTx 信号转换为 INTx 中断消息报文，并向 RC 提交中断请求。在 PCIe 总线中，共有 8 种 INTx 中断消息报文，见表 6-7。

表 6-7 INTx 中断消息报文

名 称	Code[7:0]	Routing r[2:0]	Requester ID	描 述
Assert_INTA	0010 0000	100	包括总线号和设备号，功能号保留	置 INTA#信号有效。注意在 PCIe 总线中，并没有物理上的 INTA#信号
Assert_INTB	0010 0001	100	同上	置 INTB#信号有效
Assert_INTC	0010 0010	100	同上	置 INTC#信号有效
Assert_INTD	0010 0011	100	同上	置 INTD#信号有效
Deassert_INTA	0010 0100	100	同上	置 INTA#信号无效
Deassert_INTB	0010 0101	100	同上	置 INTB#信号无效
Deassert_INTC	0010 0110	100	同上	置 INTC#信号无效
Deassert_INTD	0010 0111	100	同上	置 INTD#信号无效

当 PCIe 设备不使用 MSI 报文向 RC 提交中断请求时，可以首先使用 Assert_INTx 报文向处理器系统提交中断请求，当中断处理完毕，再使用 Deassert_INTx 报文。这些 INTx 中断消息报文的 r[2:0] 字段为 0b100，即为 Local 消息报文。设备收到该消息报文后，将结束收到的 INTx 中断消息报文，然后产生一个新的 INTx 中断消息报文。

在一个处理器系统中，PCI 设备首先需要通过 PCIe 桥，之后可能通过多级 Switch，最终到达 RC。假设 PCI 设备使用 INTA#信号进行中断请求，但是由于中断路由表的存在，PCIe 桥可能将 INTA#信号转换为 INTB 中断消息，而这个 INTB 中断消息通过 Switch 时，可能又被 Switch 的中断路由表转换为 INTC 中断消息。因此 PCIe 设备收到 INTx 中断消息后，首先需要结束当前中断消息，之后根据中断路由表产生一个新的 INTx 中断消息，直到这个中断消息传递到 RC。

2. 错误消息报文

在第 4.3.3 节中简要介绍了 AER Capability 结构。如果 PCIe 设备支持 AER Capability 结构，当 PCIe 设备出现某种错误时，将向 RC 或者 RC Event Collector 发送错误消息报文，之后 RC 或者 RC Event Collector 将根据错误类型分别进行处理。

PCIe 总线规范定义了两大类错误类型，分别是可恢复错误（Correctable Errors）和不可恢复错误（Uncorrectable Errors），不可恢复错误又细分为致命错误（Fatal）和非致命错误（Nonfatal）。当 PCIe 设备出现这些错误时，将使用 ERR_COR、ERR_NONFATAL 和 ERR_FATAL 错误消息报文向 RC 或者 RC Event Collector 发送错误消息报文。

PCIe 总线规范并没有详细描述“可恢复错误”和“不可恢复错误”的具体处理方法，也没有详细描述 PCIe 设备的错误恢复机制。对于 PCIe 设备，这些处理方法并不重要。PCIe 总线定义 AER 机制的主要考虑是，由 PCIe 设备将错误信息“统一报告”给 RC 或者 RC Event Collector，并由 RC 或者 RC Event Collector“统一处理”这些错误。其中“统一报告”和“统一处理”才是 AER 机制的设计要点。

当 PCIe 设备出现某种错误后，首先将这些错误信息保留在设备的 AER Capability 结构中，之后 RC 或者 RC Event Collector 从“来自 PCIe 设备的错误信息报文”中获得相应的错误信息。为此在 RC 中设置了一个“Error Source Identification”寄存器保存究竟是哪个 PCIe 设备发出的错误信息报文，之后 RC 或者 RC Event Collector 向处理器系统提交中断请求，由相应的中断服务例程统一处理所有 PCIe 设备的错误信息。

由 RC 统一处理所有 PCIe 设备错误信息的这种做法，势必加大 RC 和系统软件的设计复杂度。而外部设备的多样性与复杂程度决定了使用这种方法不一定能够取得较好的效果。目前 Intel 的 Chipset 已经支持 AER 机制，但是绝大多数 PCIe 设备并不支持 AER 机制。AER 机制是 Intel 统一外部设备错误处理的一种方法，这为 PCIe 设备的设计提出了更高的要求，而这种方法是否能够取得理想的效果，仍需观察。

6.3.5 PCIe 总线的原子操作

PCIe V2.1 总线规范引入原子操作的概念，原子操作仅能在存储器访问中使用。其中 RC 和 EP 可以作为原子操作的请求者和接收者，而 Switch 和多端口 RC 支持原子操作的转发。PCIe 总线支持三类原子操作，分别是 EP-to-EP，EP-to-RC 和 RC-to-EP 的原子操作。

原子操作有利于提高智能设备^①之间以及智能设备与处理器之间的数据传递效率。当智能设备与处理器进行数据交换时，将不可避免地使用某种锁机制访问临界资源。传统的做法是使用“带锁的”存储器总线事务实现这些锁机制。而使用原子操作可以在很大程度上降低“带锁的”存储器读写请求 TLP 的使用，从而提高 PCIe 总线的使用效率。

PCIe 设备使用一次原子操作可以实现之前需要多次数据操作才能完成的数据交换任务，除此之外 PCIe 设备使用原子操作还可以避免使用带锁的 PCIe 总线事务。原子操作的基本过程如下所示。

- 1) 源设备向目标设备发送原子操作请求 TLP。原子操作请求 TLP 使用 Non-Posted 方式进行数据传递，且使用基于地址的路由方式。

- 2) 当目标设备收到这个原子操作请求 TLP 之后，将从这个 TLP 指定的存储器空间中读取原始数据。

- 3) 目标设备将“原始数据”与“原子操作请求 TLP 中包含的操作数”进行某种规定的运算后产生一个新的数据。这一过程不可被其他总线事务中断，PCIe 设备保证这一过程为原子操作。这个步骤对于原子操作至关重要，也是原子操作的实现要点。

- 4) 当上述原子操作执行完毕后，目标设备使用原子操作完成报文向源设备传送数据，并将这个新的数据写入目标设备的存储器空间中。原子操作完成报文与存储器读完成的传递方式类似。

由以上分析，可以发现所谓原子操作是指 PCIe 设备“读取原始数据”“运算”和“产生新的数据”这三个过程不可被其他操作打断。这三个过程将在目标设备中一次完成，并由目标设备的硬件逻辑保证这三个过程不会被其他 TLP 干扰。

由上文所述，一个完整的原子操作由原子操作请求 TLP 和完成 TLP 组成。其中原子操作请求 TLP 的报文头与存储器请求 TLP 类似，如图 6-8 所示。原子操作请求 TLP 具有 Data Payload 字段，在 Data Payload 中包含原子操作请求 TLP 使用的操作数。

目前，PCIe 总线共支持 3 种原子操作，分别为 FetchAdd、Swap 和 CAS 原子操作。不同的原子操作使用的操作数个数并不相同，其中 FetchAdd 和 Swap 原子操作使用一个操作数，

① 智能设备中含有一个功能较强的处理器，目前高端显卡、网卡上都包含一个处理器，这类设备都可以被称为智能设备。

而 CAS 原子操作使用两个操作数。

1. FetchAdd 操作

FetchAdd 操作支持 32b 或者 64b 的操作数。如果该 TLP 的 Length 字段为 1DW 时，操作数的长度为 32b；如果该 TLP 的 Length 字段为 2DW 时，操作数的长度为 64b。FetchAdd 操作的执行过程如下所示。

- 1) PCIe 设备从 TLP 的指定 PCI 总线地址中获得原始数据。
- 2) 将原始数据与 TLP 中的操作数相加，并得到一个新的数据。相加的结果忽略进位与溢出位。
- 3) 将这个新的数据写入 TLP 指定的 PCI 总线地址中。
- 4) 使用完成报文返回指定 PCI 总线地址中的原始数据。

2. Swap 总线事务

Swap 操作也支持 32b 或者 64b 的操作数，其原则与 FetchAdd 操作完全一致。Swap 操作的执行过程如下所示。

- 1) PCIe 设备从 TLP 指定的 PCI 总线地址中读取原始数据。
- 2) 将 TLP 中的操作数写入 TLP 指定的 PCI 总线地址。
- 3) 使用完成报文返回 PCI 总线地址中的原始数据。

3. CAS 总线事务

CAS 操作支持 32b、64b 或者 128b 的操作数。如果该 TLP 的 Length 字段为 2DW 时，操作数的长度为 32b；如果该 TLP 的 Length 字段为 4DW 时，操作数的长度为 64b；如果该 TLP 的 Length 字段为 8DW 时，操作数的长度为 128b。CAS 总线事务含有两个操作数，分别为“Compare”和“Swap”。CAS 操作的执行过程如下所示。

- 1) PCIe 设备从 TLP 指定的 PCI 总线地址中获得原始数据。
- 2) 将原始数据与“Compare”操作数进行比较。
- 3) 如果结果相等，则将“Swap”操作数写入 TLP 指定的位置。
- 4) 使用完成报文返回 PCI 总线地址中的原始数据。

智能设备之间以及智能设备与处理器之间如果需要使用“Spin Lock”操作时，可以使用 CAS 原子操作实现。值得注意的是，在 x86 处理器的指令集中，也含有 CAS 类指令，该指令是实现“Spin Lock”的基础。但是在处理器系统中使用的“Spin Lock”操作与智能设备使用的“Spin Lock”操作在实现上有所不同。

6.3.6 TLP Processing Hint

当 TLP 的 TH 位为 1 时，表示在当前 TLP 中包含 Processing Hint 字段，PH 字段由 PCIe V2.1 总线规范引入。该字段的引入可以使目标设备根据源设备对数据的使用情况，合理地安排数据缓冲，从而降低 PCIe 设备的访问延时，并最大化地利用 PCIe 设备中的数据缓冲。

Processing Hint 字段的产生与智能设备的大量涌现密切相关。在智能设备中，含有一个运算能力相当强的处理器。智能设备与处理器之间的数据交换，实质上等效于两个处理器系统之间的数据传递。有些智能设备，如在显卡中使用的 GPU（Graphic Processing Unit）和 GP-GPU（General Purpose GPU）的处理能力甚至超过多数通用处理器。智能设备与处理器系统可以采用图 6-14 所示的拓扑结构连接。

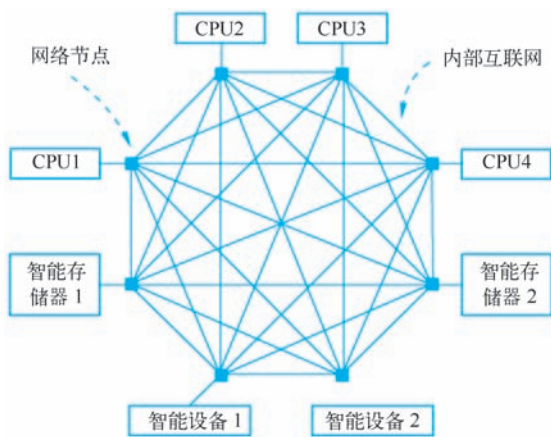


图 6-14 智能设备与处理器系统的连接

在这种处理器系统中，内部互连网络处于核心地位，上图所示的网络是一个理想的全互连结构。在这种互连结构中，处理器、存储器和智能设备与网络节点相连。在这种结构中，在网络节点上连接的设备都含有一个处理器，包括存储器。

当智能设备与处理器进行通信时，如果能够预知数据的使用情况，无疑能够减低数据的访问延时。如智能设备 1 将一组数据写入智能存储器 1 之后，如果这个智能存储器 1 能够预测这组数据将很快被再次使用，则可以将这组数据放入到高速缓冲中，而不必放入低速缓冲中。当这组数据被再次使用时，智能存储器 1 可以很快将数据从高速缓冲中读出，从而缩短了访问延时。

PCIe 总线引入了 PH 字段的主要目的是为了加速外部设备访问主存储器，即 DMA 操作，同时也兼顾 SMP 处理器系统对 PCIe 设备的访问。但是 PH 字段仍然没有完全包含上述模型的所有内容，因为在上述模型中，智能存储器是独立与 PCIe 体系结构的 RC 的，而且智能外设之间具有独立连接通路，在这种模型中，芯片内部的互连网络是真正的设计核心。目前在 P4080 处理器和 Boxboro-EX 处理器系统中，具有全互连网络结构。

1. PH 字段

PCIe 总线使用 PH 字段，使智能设备或者处理器提前预知数据的使用方法。PH 字段仅在与存储器访问相关的 TLP 中出现，该字段由两位组成，在 TLP 中的位置如图 6-8 所示，其详细说明如表 6-8 所示。

表 6-8 PH 字段

PH[1:0]	Processing Hint	描 述
00	双向数据结构	表示该 TLP 中的数据，经常被源设备和目标设备使用
01	Requester	表示该 TLP 中的数据，经常被源设备使用
10	Target	表示该 TLP 中的数据，经常被目标设备使用
11	Target with Priority	与“Target”类似，但是级别更高

当 PH[1:0] 为 0b01 时，表示 TLP 中的数据经常被设备使用，包括以下四种类型。

- DWDW (Device Write after Device Write)。外部设备对一段数据进行写操作后，很快

还会再次进行写操作。

- DWDR (Device Read after Device Write)。外部设备对一段数据进行写操作后，很快还会对这段数据进行读操作。
- DRDW (Device Write after Device Read)。外部设备对一段数据进行读操作后，很快还会对这段数据进行写操作。
- DRDR (Device Read after Device Read)。外部设备对一段数据进行读操作后，很快还会再次进行读操作。

当 PH[1:0] 为 0b10 时，表示 TLP 中的数据经常被目标设备使用，包括以下两种类型。在进行 DMA 操作时，HOST 处理器为目标设备，本节也以此为例说明 PH 字段的使用规则。

- DWHR (Host Read after Device Write)。外部设备对一段数据进行写操作后，Host 处理器将很快读取这段数据。
- HWDR (Device Read after Host Write)。Host 处理器对一段数据进行写操作后，外部设备将很快读取这段数据。

2. Steering Tag

通过上文对 PH 字段的描述，可以发现 PH 字段提供的 Processing Hint 控制能力较弱，仅是一个粗颗粒的控制机制。为此 PCIe 总线规范提供了一个 16 位的 ST (Steering Tag) 字段，目标设备通过 TLP 中的 Steering Tag 字段可以获得较为详细的信息。

当 TH 位有效时，存储器写请求 TLP 的 Tag 字段被重新定义为 ST[7:0] 字段，因为存储器写请求并不需要 Tag 字段；而对于存储器读请求 TLP，TH 位有效时 DW BE 字段被重新定义为 ST[7:0] 字段，由于一些存储器读请求 TLP 仍然需要使用 DW BE 字段处理对界，因此 ST 字段只能应用于不需要对界的存储器读请求 TLP。

当 TH 位有效时，这些“不需要对界”的存储器读请求 TLP，将使用默认的 DW BE 值。如果该存储器读请求 TLP 的 Length 字段为 1 时，First DW BE 字段的默认值为 0b1111，而 Last DW BE 字段的默认值为 0b0000；如果 Length 字段不为 1，First 和 Last DW BE 的默认值都为 0b1111。

TLP 还可以支持 ST[15:8] 字段，该字段是 ST 的扩展字段。如果一个 TLP 需要使用 ST[15:8] 字段，必须使用 TLP Prefix，因为在 TLP 头中已经没有足够的空间放置这些字段。该 TLP Prefix 也被称为 TPH TLP Prefix，其格式如图 6-15 所示。



图 6-15 TPH TLP Prefix 格式

如上图所示，在 TPH TLP Prefix 的 Byte 1 中存放 ST[15:8] 字段。在 PCIe 总线中，ST[15:8] 字段是可选的，实际上整个 ST 字段都是可选的。TPH Requester Capability 结构使用 ST Mode Select 字段定义了 ST 字段的三种使用模式。

- 当该字段为 0b000 时，表示当前 PCIe 设备不支持 ST 字段，此时 TLP 的 ST 字段为

全 0。

- 当该字段为 0b001 时，表示 ST 模式为“Interrupt Vector Mode”。此时 TLP 的 ST 字段由 MSI/MSI-X 的中断向量号确定。
- 当该字段为 0b010 时，表示 ST 字段由 PCIe 设备决定。

PCIe 设备可以根据 TLP 的属性，决定 ST 字段的值，为此在一个 PCIe 设备中，将使用 ST 表，存放这个设备使用的所有 ST 字段。这个 ST 表可以存放在 TPH Requester Capability 结构中，也可以存放在 MSI-X 表中。实际上 ST 表的存放位置并不重要，只要 PCIe 设备能够根据发出的 TLP 类型，选择合适的 ST 字段即可。

目前尚无支持 ST 字段的 PCIe 设备，这些 PCIe 设备发出的 TLP 中都不包含 ST 字段。而从 PCIe 总线规范 V2.1 中，可以发现 MSI/MSI-X 中断请求可以使用 ST 字段，当 PCIe 设备使用 MSI 或者 MSI-X 中断请求时，可以根据中断向量的不同，从 ST 表中 MSI 报文[⊖]选择合适的 ST 字段，然后再发向处理器系统。处理器系统收到这个 MSI 报文后，可以根据 ST 字段的的不同，分别处理 PCIe 设备发出的中断请求。

综上所述，ST 字段的主要目的是细分 TLP 的属性，处理器系统可以使用该字段优化数据缓冲，减小数据访问延时。ST 字段的支持需要多个 PCIe 设备共同参与。如 EP 进行 DMA 写操作时，数据将发向 RC，RC 和 EP 都需要具有解释这个 TLP 所携带的 ST 字段的能力。

因此处理器系统在初始化 PCIe 设备时，需要合理地设置该设备的 ST 表。目前尚无支持 TPH 位和 ST 字段的 PCIe 设备，但是这种方法可以有效降低 PCIe 设备访问存储器，以及 PCIe 设备间数据访问的延时，从而提高 PCIe 链路的利用率。

6.4 TLP 中与数据负载相关的参数

在 PCIe 总线中，有些 TLP 含有 Data Payload，如存储器写请求、存储器读完成 TLP 等。在 PCIe 总线中，TLP 含有的 Data Payload 大小与 Max_Payload_Size、Max_Read_Request_Size 和 RCB 参数相关。下面将分别介绍这些参数的使用。

6.4.1 Max_Payload_Size 参数

PCIe 总线规定在 TLP 报文中，数据有效负载的最大值为 4 KB，但是 PCIe 设备并不一定能够发送这么大的数据报文。PCIe 设备含有“Max_Payload_Size”和“Max_Payload_Size Supported”参数，这两个参数分别在 Device Capability 寄存器和 Device Control 寄存器中定义，这两个寄存器在 PCI Express Capability 结构中的位置见第 4.3.2 节。

“Max_Payload_Size Supported”参数存放一个 PCIe 设备中 TLP 有效负载的最大值，该参数由 PCIe 设备的硬件逻辑确定，系统软件不能改写该参数。而 Max_Payload_Size 参数存放 PCIe 设备实际使用的 TLP 有效负载的最大值。该参数由 PCIe 链路两端的设备协商决定，是 PCIe 设备进行数据传送时实际使用的参数。

PCIe 设备发送数据报文时，使用 Max_Payload_Size 参数决定 TLP 的最大有效负载。当

⊖ MSI 或者 MSI-X 中断请求使用存储器写请求 TLP。

PCIe 设备的所传送的数据大小超过 Max_Payload_Size 参数时，这段数据将被分割为多个 TLP 进行发送。当 PCIe 设备接收 TLP 时，该 TLP 的最大有效负载也不能超过 Max_Payload_Size 参数，如果接收的 TLP，其 Length 字段超过 Max_Payload_Size 参数，该 PCIe 设备将认为该 TLP 非法。

RC 或者 EP 在发送存储器读完成 TLP 时，这个存储器读完成 TLP 的最大 Payload 也不能超过 Max_Payload_Size 参数，如果超过该参数，PCIe 设备需要发送多个读完成报文。值得注意的是，这些读完成报文需要满足 RCB 参数的要求，有关 RCB 参数的详细说明见下文。

在实际应用中，尽管有些 PCIe 设备的 Max_Payload_Size Supported 参数可以为 256 B、512 B、1024 B 或者更高，但是如果 PCIe 链路的对端设备可以支持的 Max_Payload_Size 参数为 128 B 时，系统软件将使用对端设备的 Max_Payload_Size Supported 参数，初始化该设备的 Max_Payload_Size 参数，即选用 PCIe 链路两端最小的 Max_Payload_Size Supported 参数初始化 Max_Payload_Size 参数。

在多数 x86 处理器系统的 MCH 或者 ICH 中，Max_Payload_Size Supported 参数为 128 B。这也意味着在 x86 处理器中，与 MCH 或者 ICH 直接相连的 PCIe 设备进行 DMA 读写时，数据的有效负载不能超过 128 B，同时读完成携带的 Payload 也不能超过 128 B。而在 PowerPC 处理器系统中，该参数大多为 256 B。

目前在大多数 EP 中，Max_Payload_Size Supported 参数不大于 512 B，因为在大多数处理器系统的 RC 中，Max_Payload_Size Supported 参数也不大于 512 B。因此即便 EP 支持较大的 Max_Payload_Size Supported 参数，并不会提高数据传送效率。

而 Max_Payload_Size 参数的大小与 PCIe 链路的传送效率成正比，该参数越大，PCIe 链路带宽的利用率越高，该参数越小，PCIe 链路带宽的利用率越低。

PCIe 总线规范规定，对于实时性要求较高的 PCIe 设备，Max_Payload_Size 参数不应设置过大，因此这个参数有时会低于 PCIe 链路允许使用的最大值。

6.4.2 Max_Read_Request_Size 参数

Max_Read_Request_Size 参数由 PCIe 设备决定，该参数规定了 PCIe 设备一次能从目标设备读取多少数据。

Max_Read_Request_Size 参数在 Device Control 寄存器中定义，详见第 4.3.2 节。该参数与存储器读请求 TLP 的 Length 字段相关，其中 Length 字段不能大于 Max_Read_Request_Size 参数。在存储器读请求 TLP 中，Length 字段表示需要从目标设备读取多少数据。

值得注意的是，Max_Read_Request_Size 参数与 Max_Payload_Size 参数间没有直接联系，Max_Payload_Size 参数仅与存储器写请求和存储器读完成报文相关。

PCIe 总线规定存储器读请求，其读取的数据长度不能超过 Max_Read_Request_Size 参数，即存储器读 TLP 中的 Length 字段不能大于这个参数。如果一次存储器读操作需要读取的数据范围大于 Max_Read_Request_Size 参数时，该 PCIe 设备需要向目标设备发送多个存储器读请求 TLP。

PCIe 总线规定 Max_Read_Request_Size 参数的最大值为 4 KB，但是系统软件需要根据硬

件特性决定该参数的值。因为 PCIe 总线规定 EP 在进行存储器读请求时，需要具有足够大的缓冲接收来自目标设备的数据。

如果一个 EP 的 Max_Read_Request_Size 参数被设置为 4 KB，而且这个 EP 每发出一个 4 KB 大小存储器读请求时，EP 都需要准备一个 4 KB 大小的缓冲^①。这对于绝大多数 EP 都是一个相当苛刻的条件。为此在实际设计中，一个 EP 会对 Max_Read_Request_Size 参数的大小进行限制。

6.4.3 RCB 参数

RCB 位在 Link Control 寄存器中定义，见第 4.3.2 节。RCB 位决定了 RCB 参数的值，在 PCIe 总线中，RCB 参数的大小为 64B 或者 128B，如果一个 PCIe 设备没有设置 RCB 的大小^②，则 RC 的 RCB 参数缺省值为 64B，而其他 PCIe 设备的 RCB 参数的缺省值为 128B。PCIe 总线规定 RC 的 RCB 参数的值为 64B 或者 128B，其他 PCIe 设备的 RCB 参数为 128B。

在 PCIe 总线中，一个存储器读请求 TLP 可能收到目标设备发出的多个完成报文后，才能完成一次存储器读操作。因为在 PCIe 总线中，一个存储器读请求最多可以请求 4KB 大小的数据报文，而目标设备可能会使用多个存储器读完成 TLP 才能将数据传递完毕。

当一个 EP 向 RC 或者其他 EP 读取数据时，这个 EP 首先向 RC 或者其他 EP 发送存储器读请求 TLP；之后由 RC 或者其他 EP 发送存储器读完成 TLP，将数据传递给这个 EP。

如果存储器读完成报文所传递数据的地址范围没有跨越 RCB 参数的边界，那么数据发送端只能使用一个存储器完成报文将数据传递给请求方，否则可以使用多个存储器读完成 TLP。

假定一个 EP 向地址范围为 0xFFFF-0000~0xFFFF-0010 的这段区域进行 DMA 读操作，RC 收到这个存储器读请求 TLP 后，将组织存储器读完成 TLP，由于这段区域并没有跨越 RCB 边界，因此 RC 只能使用一个存储器读完成 TLP 完成数据传递。

如果存储器读完成报文所传递数据的地址范围跨越了 RCB 边界，那么数据发送端（目标设备）可以使用一个或者多个完成报文进行数据传递。数据发送端使用多个存储器读完成报文完成数据传递时，需要遵循以下原则。

- 第一个完成报文所传送的数据，其起始地址与要求的起始地址相同。其结束地址或者为要求的结束地址（使用一个完成报文传递所有数据），或者为 RCB 参数的整数倍（使用多个完成报文传递数据）。
- 最后一个完成报文的起始地址或者为要求的起始地址（使用一个完成报文传递所有数据），或者为 RCB 参数的整数倍（使用多个完成报文传递数据）。其结束地址必须为要求的结束地址。
- 中间的完成报文的起始地址和结束地址必须为 RCB 参数的整数倍。

当 RC 或者 EP 需要使用多个存储器读完成报文将 0xFFFFE-FFF0~0xFFFF-00C7 之间的数

① 这是流量控制 Infinite FC Unit 的要求，详见第 9.3.2 节。

② 有些 PCIe 设备可能没有 Link Control 寄存器。

据发送给数据请求方时，可以将这些完成报文按照表 6-9 方式组织。

表 6-9 存储器读完成报文的拆分方法

方式 1	方式 2	方式 3
0xFFFFE-FFF0~0xFFFFE-FFFF	0xFFFFE-FFF0~0xFFFFE-FFFF	0xFFFFE-FFF0~0xFFFFE-FFFF
0xFFFFF-0000~0xFFFFF-003F	0xFFFFF-0000~0xFFFFF-007F	0xFFFFF-0000~0xFFFFF-00C7
0xFFFFF-0040~0xFFFFF-007F	0xFFFFF-0080~0xFFFFF-00C7	
0xFFFFF-0080~0xFFFFF-00BF		
0xFFFFF-00C0~0xFFFFF-00C7		

上表提供的方式仅供参考，目标设备还可以使用其他拆分方法发送存储器读完成 TLP。PCIe 总线使用多个完成报文实现一次数据读请求的主要原因是考虑 Cache 行长度和流量控制。在多数 x86 处理器系统中，存储器读完成报文的数据长度为一个 Cache 行，即一次传送 64B。除此之外，较短的数据完成报文占用流量控制的资源较少，而且可以有效避免数据拥塞。有关流量控制的内容详见第 9 章。

6.5 小结

本章重点介绍 PCIe 总线的事务层。在 PCIe 总线层次结构中，事务层最易理解，同时也与系统软件直接相关。但是事务层的知识较为琐碎，在第 12 章将结合一个 EP 的设计实例，进一步说明 PCIe 总线事务层的具体实现机制。